ACS — TR-069 Auto Configuration Server

UAT Readiness Report

What has been built · How it runs day to day · What comes next · What still blocks UAT

Executive summary

ACS is a multi-tenant TR-069 device-management platform: it onboards customer-premises equipment the moment it first calls home, keeps its configuration to a defined standard, pushes firmware in controlled waves, and gives an operator one console from which to see and act on the whole fleet. This report updates the assessment of 9 September 2026 against revision **d2ea8d1**, twenty-one commits later, and adds an appendix of real screens from the running system.

The verdict is unchanged in shape but better in substance: the platform is ready for a controlled lab UAT and not yet for a subscriber-facing one. Two of the five previously recorded blockers have been closed by engineering since the last report. What still stands between today and testers is environment, hardware and network access — not software — plus a small number of defects this run surfaced by exercising the product rather than reading it.

Readiness at a glance

Area	State	Comment
Build, unit and integration tests	**Pass**	25 Go packages and 52 frontend tests green, including the database-backed suite against a live PostgreSQL 18.
Zero-touch device onboarding	**Proven live**	Eight CPE across five vendors onboarded over TR-069 with Digest authentication, unattended.
Job queue and remote actions	**Proven live**	Reboot, schedule-inform and parameter reads dispatched and completed; failures recorded with a reason.
Configuration templates and policies	**Proven live**	A template auto-applied itself to a matching device on its next session, without an operator touching it.
Monitoring and dashboards	**Proven live**	Prometheus scraping all three services; three provisioned Grafana dashboards showing real fleet data.
Documented quick start	**Fixed since last report**	The command that failed in the previous assessment now runs. Previously blocker B1.
Console reachable from its own API	**New defect**	The documented start script leaves the API's allowed browser origin at localhost, so a console served on a host address cannot call it. Blocker B6.

Area	State	Comment
Real CPE hardware qualified	**None**	Still zero recorded real-device results. Blocker B2, and still the single highest-value action.
TLS on the CWMP interface	**Not enabled**	Device credentials would cross the network in clear text during UAT. Blocker B3.
Scale	**Unmeasured**	The harness now counts transport failures honestly, but no figure has been taken on Linux. Blocker B4.
Defined UAT environment	**Outstanding**	Database, network restriction, secret custody and log retention are still administrator decisions. Blocker B5.

Overall: the product works, and this report shows it working. The remaining engineering effort is roughly two to three days; the rest of the critical path belongs to the administrators, and the items with the longest lead times — test CPE, a TLS certificate and DNS name, and firewall rules — should be requested today.

1. What has been built

ACS is a full device-management platform: roughly 28,000 lines of Go across three services and 15,000 lines of TypeScript in a nineteen-screen operator console, on a 52-migration PostgreSQL schema behind a documented 105-path API.

System shape

Component	Interface	Responsibility
CWMP gateway	7547, 3478/udp	Terminates CPE sessions with Digest, Basic or mutual-TLS authentication; dispatches queued work one remote call at a time; reaps stale job leases and dead devices; answers STUN so devices behind carrier-grade NAT can still be reached.
Operator API	8080	105 documented REST paths behind sign-in, four permission tiers and centrally-enforced tenancy scoping; also hosts the connection-request dispatcher, the scheduled-job worker and the retention pruner.
BSS adapter	8090	The contract a CRM or billing system integrates against: client-credential authentication, account-to-device mapping, idempotent order dispatch and signed webhooks with backoff.
Operator console	Web	React 19 and TypeScript, nineteen screens, an in-app handbook, and an accessibility-tested design system.
Observability	Prometheus	Forty metric series, alert routing, and three provisioned Grafana dashboards for fleet, integration and tenancy views.

What an operator can actually do today

Capability area	Delivered function
Device lifecycle	Zero-touch onboarding from the first call home, automatic template application, parameter discovery, labels and map coordinates, tags and groups, and bulk import from spreadsheet or XML.
Configuration	Read and write device parameters, add and remove objects, reusable configuration templates applied in bulk or automatically, and continuous policy compliance evaluated on every check-in.
Firmware	Signed expiring download links, staged rollouts with a canary percentage, a maximum-failure-rate brake, rollback images, and single-device pushes.
Remote actions	Reboot, factory reset, schedule a check-in, ping and traceroute diagnostics, and connection requests over direct addressing or STUN, falling back to the next scheduled check-in.
Fleet operations	Bulk actions across a filtered selection, recurring scheduled jobs, a durable job queue with leases and dead-lettering, and fleet-health dashboards.
Multi-tenancy	A region and customer hierarchy, per-operator scopes, four permission tiers, deny-by-default for unscoped operators, and a full audit log.
Device access	A browser-based console to the device over SSH or Telnet with pinned host keys, a reverse proxy to the device's own web interface behind an allowlist, and a VPN peer registry.
Integration	Order dispatch and job-status passthrough for a BSS or CRM, webhook subscriptions, spreadsheet export, and two published API contracts that a test fails on if the code drifts from them.

Engineering practices already in place

These reduce UAT risk materially and are stronger than is typical at this stage, so they are worth stating plainly:

- **Fail-closed configuration.** A missing, placeholder or too-short secret stops a service starting rather than producing a warning — and a dedicated build job proves placeholder secrets are refused.

- **Contract enforcement.** A test fails if the published API specification drifts from the routes actually registered, and the console's types are regenerated and compared on every build.
- **A comprehensive build pipeline** covering formatting, static analysis, race-enabled tests, vulnerability scanning, a clean-database migration job that also proves repeat runs and concurrent runners are safe, container builds with a software bill of materials, image scanning and secret scanning.
- **A committed mock-CPE harness** covering four vendor profiles, transfers, malformed messages and a load mode — so device behaviour is a regression-tested fixture rather than a hand-run script.
- **Rate limiting that actually fires.** Both the API and the CWMP listener refused this report's own scripted traffic when it exceeded the configured rate — the protection is real, not nominal.

2. Who uses it, and what each person sees

Four audiences, four surfaces. Each was captured live for Appendix A.

Role	Where they work	What they see
NOC operator	Console: Dashboard, Fleet Health, Jobs	Which devices are online, what work is outstanding, and what failed overnight — with the reason attached to each failure rather than a bare status.
Field or support engineer	Console: Device Fleet and one device in detail	One subscriber's device: its parameters, its history, and the actions that fix it — reboot, re-apply configuration, open a console to it, or run a diagnostic.
Provisioning / fleet manager	Console: Templates, Policies, Scheduled Jobs, Rollouts	The standard a device must meet, applied automatically to new arrivals, and firmware moved through the fleet in controlled waves.
Platform administrator	Console: Operators, Tenancy, BSS Integration, Audit Log; plus Grafana	Who may do what and to which part of the estate, what the integrated business systems are doing, and the health of the platform itself.

The console is deliberately one application with a permission model behind it rather than four separate tools: a scoped operator sees the same screens, narrowed to the devices they are entitled to. An operator with no scope sees nothing at all — the default is deny, not allow.

3. How it works, day to day

This section describes the ordinary rhythm of the platform in service. Every step below was performed against the running system while this report was being written.

A device arrives: onboarding without anybody touching it

A CPE is configured — at the factory, by the installer, or by the previous management server — with the ACS address and a credential. The first time it powers on, it calls home. The platform authenticates it, creates its record, reads back what it reported about itself, and files it under the right customer. Nobody presses anything. Eight devices across five vendors did exactly this for this report:

```
Huawei HW5G2611000042 inform=200 close=204 InformResponse=True
Nokia NK3G24A9F00117 inform=200 close=204 InformResponse=True
Teltonika TL1103992288 inform=200 close=204 InformResponse=True
Zyxel S230Q12345678 inform=200 close=204 InformResponse=True
ZOWEE ZW5G0007741 inform=200 close=204 InformResponse=True

8/8 devices completed a CWMP session
```

If a configuration template matches the model, it applies itself on that same session — the device is brought to standard before an operator has seen it. That happened live here: two devices matching the template's model filter were sent their settings automatically on their next check-in.

The NOC's morning: what is wrong, and what failed overnight

The operator opens the console to a dashboard of fleet state (Figure A2) and a job list (Figure A6). The job list is the important one: it is not a queue of pending work, it is a record of what the platform tried to do and how it went. A failure carries its reason — in the live run, two diagnostics failed with *“response did not match the RPC that was sent”*, which tells an engineer precisely where to look rather than leaving them to guess.

Devices that stop checking in are marked offline automatically by a background reaper rather than lingering as stale green rows — visible in this report's own capture, where the fleet moved from online to offline once the mock devices stopped calling home.

A subscriber calls: one device, in detail

The engineer searches by serial, label, model or vendor and opens the device (Figure A18). From there they can read the current parameter values, compare against what the device reported previously, push a corrected setting, reboot it, run a ping or traceroute from the device itself, or — when the fault needs it — open a terminal or the device's own web interface through the platform, without a VPN to the customer's premises and without the device being exposed to the internet.

Situation	What the operator does
Wrong Wi-Fi settings after a customer reset	Re-apply the configuration template. The correct values are pushed on the device's next contact.
Device unreachable to instant commands	The platform falls back to the device's next scheduled check-in, so the command still lands — later, but without an engineer visit.
Suspected line problem	Run ping and traceroute from the device and read the result in the console.
Fault needing the device's own interface	Open the device web interface or a terminal session through the platform, with the access recorded in the audit log.

Provisioning: keeping the fleet to a standard

Two mechanisms do most of the work. A **template** is a named set of settings applied to a device or a whole group — and, if the operator chooses, to every future device matching a model. A **policy** is a rule the platform checks on every single check-in: if a device drifts from the required value, it shows as non-compliant without anybody running a report. Both were created live for this report and both are visible in Figures A9 and A11.

Firmware: moving a fleet without breaking it

A rollout goes out in waves. A small canary group upgrades first; if failures exceed a set rate the rollout stops itself rather than continuing into the fleet. Download links are signed and expire, so a firmware image is not left publicly fetchable, and a rollback image can be nominated in advance.

The business systems: orders arriving from the CRM

The BSS adapter is the contract another system integrates against. An order arrives, is mapped to the right device, and is dispatched idempotently — the same order sent twice does not act twice. Job status flows back, and subscribed systems receive signed webhooks with retry and backoff. The console carries a troubleshooting panel for this path (Figure A14) so an integration problem is diagnosed in the product rather than in a log file.

Oversight: the monthly and the continuous view

Every privileged action is recorded in an audit log (Figure A16) — who did what, to which device, and when. Reports export to a spreadsheet for people who live in one (Figure A15). And the platform's own health is on three Grafana dashboards (Figures A19 to A21): fleet state and check-in rate, integration throughput, and a tenancy view for locating a customer's devices.

Running quietly in the background

- **Stale-lease recovery.** Work claimed by a session that then died is returned to the queue, or dead-lettered if it must not be repeated.
- **Liveness reaping.** Devices that stop checking in are marked offline on a schedule.
- **Retention.** Sessions, audit entries, parameter history and finished jobs are pruned on a defined schedule — with one table currently failing, recorded as defect B7.
- **Alerting.** Alert rules and routing exist and fire; they currently page nobody, because no destination has been agreed yet.

4. Built, but deliberately not switched on

Several capabilities exist in the codebase but are deliberately inactive. They are listed here so that their absence during UAT is understood as a decision rather than discovered as a defect.

Capability	State	Why it is off, and what turning it on would need
TLS on the CWMP interface	**Built, not enabled**	The listener supports TLS and mutual TLS, and the minimum version is configurable. It needs a certificate for an agreed hostname — an administrator decision, not development work. This is blocker B3.
Mutual-TLS device authentication	**Built, not enabled**	An alternative to shared Digest credentials, appropriate where the CPE fleet can carry client certificates. Needs a certificate authority decision.
TR-369 / USP controller	**Designed, not built**	The successor protocol to TR-069. A design exists in the repository; no implementation. Out of scope for this UAT and should be stated as such in writing.
TR-098 parameter writes	**Read only**	The platform reads both the older TR-098 and the current TR-181 data models but writes only TR-181. If any device in the test fleet is TR-098-only, configuration changes will fail against it — confirm the data model of each target model before UAT.
BSS suspend and activate	**Deliberately refused**	These operations are refused pending a per-vendor walled-garden parameter. The integration contract cannot be completed until each vendor supplies it.
VPN concentrator	**Registry only**	Peers can be enrolled and addresses allocated, but the generated client configuration is incomplete until a real concentrator's public key and endpoint are configured.
Operator single sign-on and MFA	**Not supported**	Commercial platforms offer this. If it is mandatory for production sign-off it must be known now, because it is development work rather than configuration.

5. Security posture

The platform holds credentials for every device it manages and can reboot, reconfigure or factory-reset any of them. Its own security therefore matters as much as its features. The following were confirmed against the running system rather than taken from documentation.

- **Nothing starts with a weak secret.** Five secrets are mandatory; a missing, placeholder or too-short value is a fatal startup error.
- **Unauthenticated access is refused.** The API rejected both an unauthenticated request and a wrong password; a valid sign-in issued a scoped, time-limited token.
- **Devices must authenticate too.** The gateway issued a correct Digest challenge and refused a replayed authentication header.
- **Tenancy is enforced centrally, not per screen.** An operator without a scope sees nothing; a scoped operator cannot create objects outside their own customer — the platform refused exactly that during this report's own seeding.
- **Rate limiting is real.** Both the API and the CWMP listener refused this report's scripted traffic when it went too fast.
- **Device credentials are encrypted at rest** under a dedicated key, separate from the token-signing secret.
- **The dashboard database role is read-only and column-restricted**, deliberately excluding the columns that carry webhook signing keys and job payloads — so a dashboard editor cannot read a Wi-Fi passphrase out of a job's arguments.
- **Every privileged action is audited**, including terminal sessions opened to a device.
- **Only the device-facing ports are published on all interfaces.** The API, console, database and monitoring stack bind to the loopback address and expect a TLS-terminating proxy in front.

Two qualifications belong here. The CWMP interface currently runs without TLS, so device credentials would cross the network in clear text during UAT — blocker B3. And the device network allowlist is unset in this configuration, which the service itself warns about at startup: until it is set, the web-interface proxy could reach any address the host can reach.

6. What changed since the 9 September assessment

Twenty-one commits landed between the previously assessed revision and this one. Their effect on that report's findings, re-tested rather than assumed:

Previous finding	Now	Evidence
B1 — the documented quick start does not run	**Closed**	The exact command in the README was executed verbatim against this revision and started the database. The fix was made in documentation rather than by weakening the compose file's own secret requirements — the better of the two available choices.
B4 — the load harness under-reports failures	**Half closed**	The harness now counts transport failures and no longer hides them behind a fatal test error. The other half — an honest scale figure measured on Linux rather than a Windows workstation — has still not been taken.
B2 — no qualified CPE hardware	**Unchanged**	The compatibility matrix still carries four vendor profiles with no real-device result and no firmware version recorded against any of them.
B3 — TLS not enabled on CWMP	**Unchanged**	Still unticked on the hardening checklist.
B5 — no defined UAT environment	**Unchanged**	Still an administrator decision set.

Improvements that were not on the previous list

- Webhook signatures are no longer replayable, and the legacy signature header was removed outright rather than deprecated — a cleaner decision than carrying it.
- The cellular fallback parameter paths were corrected and the guarding test widened to catch signal-quality readings too.
- Wi-Fi passphrases are now written to the correct TR-098 parameter, and canonical parameters resolve through an ordered list of candidates rather than a single guess — this is the groundwork that makes multi-vendor support predictable.
- The documentation stopped claiming conformance to a webhook standard the implementation does not fully meet. Removing an overstated claim is worth as much to an assessor as adding a feature.

What this run found that no previous report records

Four defects, all found by running the product rather than reading it. They are carried into Section 7 as blockers B6 to B9.

7. Outstanding blockers to UAT

Nine items stand between today and inviting testers. They are ordered by what stops UAT starting at all, followed by what would make its results untrustworthy. Most are not engineering work.

Blocks UAT from starting

Ref	Blocker	Severity	Why it stops UAT
B2	No qualified CPE hardware	**Critical**	Zero real-device results exist. If no device has been proven to onboard, UAT fails on its first morning for reasons nobody present can diagnose. Two capabilities matter especially for a cellular fleet: connection requests over UDP for devices behind carrier-grade NAT, implemented from the specification and never tested against hardware; and whether the already-deployed ZOWEE unit ever completed onboarding, which is recorded nowhere.
B3	TLS is not enabled on the CWMP interface	**Critical**	Device credentials and the whole device conversation would cross the network in clear text throughout UAT.
B6	The console cannot reach its own API when served on a host address	**Critical**	The documented start script bakes the API address into the console for the host's public address, but sets neither the API's allowed browser origin nor the console base URL — so the browser blocks every call and the console shows *“Failed to reach the API”*. Reproduced on the first attempt in this run; this is exactly the shape of the previous report's B1, one layer further in.
B5	No defined UAT environment	**High**	Database, network restriction, secret custody and log-retention decisions are all still open.

Makes UAT results untrustworthy

Ref	Blocker	Severity	Why it matters
B4	Scale is unmeasured	**High**	The harness now counts failures honestly, but no figure has been taken on hardware resembling the target. Until it has, the fleet ceiling is an assumption.
B7	The retention pruner fails on every pass	**Medium**	One rule addresses a table by a column that does not exist, so every retention run errors and expired password-reset tokens are never removed. The failure is logged once per pass and otherwise silent.
B8	The test suite fails when run the documented way	**Medium**	Three database-backed packages share one database and race when the test runner uses its default parallelism: a newcomer running the obvious command sees migration and deadlock errors, not a clean pass. Serialising the run makes all 25 packages green.
B9	Duplicate names return a server error, not a conflict	**Low**	Creating a region or project whose name already exists produces a 500 with the message “internal error” rather than a 409. A tester will report it as a crash, and an integrator cannot distinguish it from a real fault.

B6 to B9 together are roughly two to three days of engineering. B2, B3 and B5 are not engineering at all — they are access, hardware and decisions, and they set the UAT start date.

8. What to ask the administrators

Most of what stands between the current state and a signed-off UAT is not engineering work. This section is written to be taken directly into that conversation. Lead with the three asks that have lead times — the test CPE, the certificate and DNS name, and the firewall rules — because those, not the code, determine the start date.

Network administrators

Ask	For	Why, in their terms
Inbound TCP 7547 from the CPE address ranges only	B2	The port devices call home on. Without it nothing onboards. It should be restricted to the mobile or broadband ranges the test devices sit in, never opened to the internet.
Inbound UDP 3478 to the same host	B2	The platform's own STUN responder discovers the public address of devices behind carrier-grade NAT. Without it, instant actions cannot work and every command waits for the device's next scheduled check-in.
Outbound access to the CPE connection-request ports	B2	The platform must be able to call a device back. Confirm whether the carrier path permits this at all — a “no” is a finding worth having before UAT rather than during it.
A DNS name for the ACS endpoint	B3, B6	Devices are provisioned with a management address that is painful to change once they are in the field. Agree the permanent hostname now.
Console and API restricted to office or VPN ranges	B3, B5	The management interface should never be reachable from the public internet.

Security administrators

Ask	For	Why, in their terms
A TLS certificate for the CWMP hostname	B3	Device credentials cross the network on every session. The single highest-value security item, and it gates UAT.
A decision on the minimum TLS version	B3	The platform can go as low as TLS 1.0 for legacy devices. Set the floor as high as the actual test fleet permits and treat any lowering as a per-device exception.
Secret storage and custody	B5	Five secrets are mandatory and the services refuse to start without them. Decide now whether they live in a secrets manager or a protected file, and who holds them.
An audit-log forwarding destination	B5	Every privileged action is already recorded; agreeing where those records ship makes the trail admissible.
A position on operator single sign-on and MFA	—	Not currently supported. If it is mandatory for production sign-off, that must be known now — it is development work, not a setting.
A break-glass and offboarding process	—	The highest operator tier can factory-reset devices and open terminal sessions to them. Agree who holds it and how access is revoked.

Infrastructure and database administrators

Ask	For	Why, in their terms
A managed PostgreSQL instance for UAT	B5	The container database is a development convenience. A managed instance gives UAT the backup and point-in-time-recovery behaviour production will need, and lets a restore be rehearsed for real.
A backup schedule and a restore rehearsal slot	B5	Backup and restore scripts exist but have never been exercised. Until a restore is timed, the recovery objectives are assumptions.
A Linux host sized for the target fleet	B4	Needed both to run UAT and to re-measure the load ceiling honestly — every figure taken so far came from a Windows workstation and is not a fair reading of the platform.
Storage for firmware images and device uploads	B5	Local disk is the default; object storage is supported and is the better choice if more than one instance will ever run.
Mail relay credentials	B5	Without them, operator password-reset links are written to the log instead of sent — which testers will hit the first time somebody forgets a password.
Where alerts should be delivered	B5	Alert rules and routing already exist but currently page nobody. A webhook, mailbox or chat channel turns existing monitoring into real on-call.

Fleet, vendor and business administrators

Ask	For	Why, in their terms
At least one physical CPE per vendor, with exact firmware versions	B2	The critical path. Device behaviour changes between firmware builds, so the version must be recorded alongside the model. Include the ZOWEE unit already deployed.
A dedicated test unit that may be rebooted and factory-reset	B2	Destructive paths cannot be qualified on a device carrying live service.
The management credentials provisioned on those devices	B2	Needed to point them at the UAT platform. Confirm whether the fleet uses one shared credential or one per device — both are supported, but changing later is disruptive.
Confirmation of each model's data model	B2	The platform reads both data models but writes only the current one. A device that supports only the older model cannot be configured, and that is development work rather than a setting.
The vendor answer on suspend and activate	—	Those operations are deliberately refused pending a per-vendor parameter. The integration contract cannot be completed without it.
Agreed UAT scope and exit criteria	All	Place the successor protocol, older-data-model writes, throughput diagnostics and high availability out of scope *in writing*, so their absence is not raised as a defect mid-test.

9. Recommended path to UAT

Step	Work	Owner	Duration
1	Request the test CPE, the TLS certificate and DNS name, and the firewall rules — in parallel, today. These have the longest lead times and set the start date (B2, B3).	Sponsor / administrators	Same day
2	Fix the console-to-API origin defect and prove it by loading the console on a non-localhost address (B6).	Engineering	Half a day
3	Fix the retention pruner, make the test suite pass the documented way, and return a conflict rather than a server error on duplicate names (B7, B8, B9).	Engineering	1–2 days
4	Stand up the UAT environment with TLS and restricted networking, tick the hardening checklist, and rehearse a database restore (B3, B5).	Infrastructure	On delivery
5	Qualify one real device end to end and record the compatibility row; re-run the load harness on Linux and record the figure (B2, B4).	Engineering + fleet	1–2 days
6	Open UAT, using the remaining vendor devices as the first structured test campaign.	Joint	UAT starts

Steps 2 and 3 can run while the hardware and certificates are being obtained, so the engineering work is not on the critical path. Realistically, UAT can open two to three weeks after the administrator asks in step 1 are placed — and the qualification in step 5 is the one item that must not be skipped, because it is the thing UAT would otherwise discover on its first morning.

How this compares to comparable systems

Measured against the commercial device-management platforms and against the open-source benchmark, three things stand out. Native multi-tenancy — a region and customer hierarchy with per-operator scopes and deny-by-default — is something the open-source baseline does not provide and operators typically build themselves. A first-class business-system integration contract with client-credential authentication, idempotent order dispatch and signed webhooks is usually a professional-services engagement with the commercial vendors. And the engineering discipline — fail-closed secrets, contract-drift tests, bill-of-materials generation and image scanning — is stronger than is typical at this maturity.

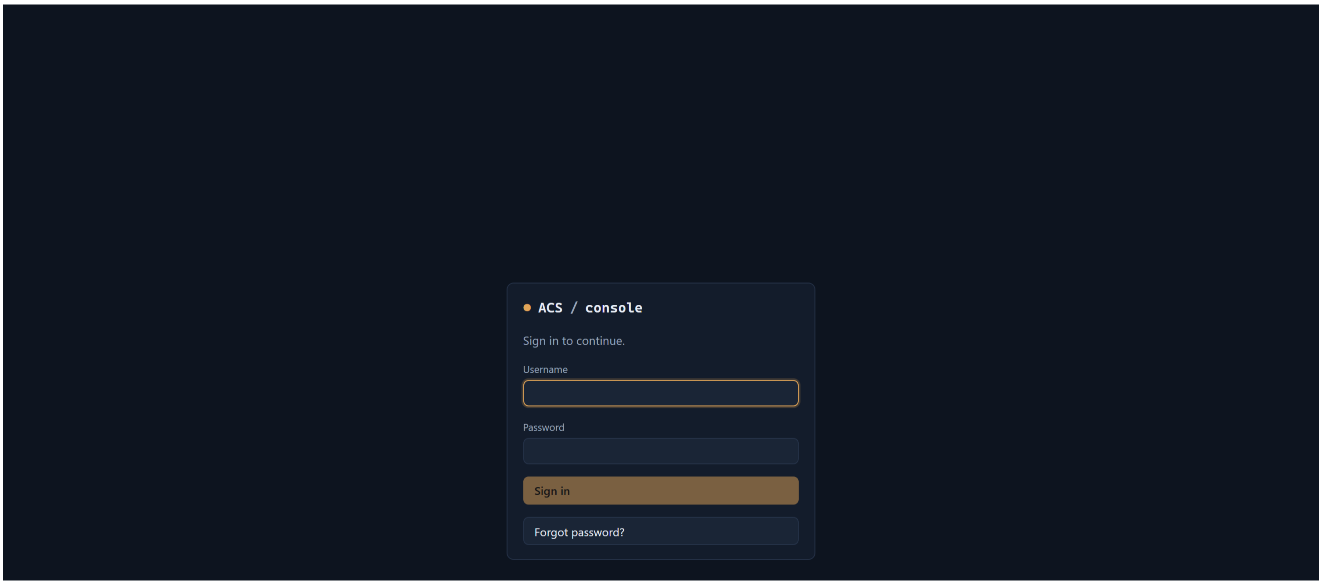
The honest gaps are hardware qualification breadth, operator single sign-on, high availability, and the successor protocol. None of these is a surprise at this stage; all of them are worth naming in the UAT scope document so they are not raised as defects.

Appendix A. The system on screen

Every figure below is a real screen captured on 10 September 2026 from the running platform at revision d2ea8d1, with a mock fleet of eight CPE across five vendors that genuinely onboarded over TR-069. The device records, the job outcomes, the audit entries and the dashboard readings are all produced by the product, not by fixtures written into the database.

Signing in

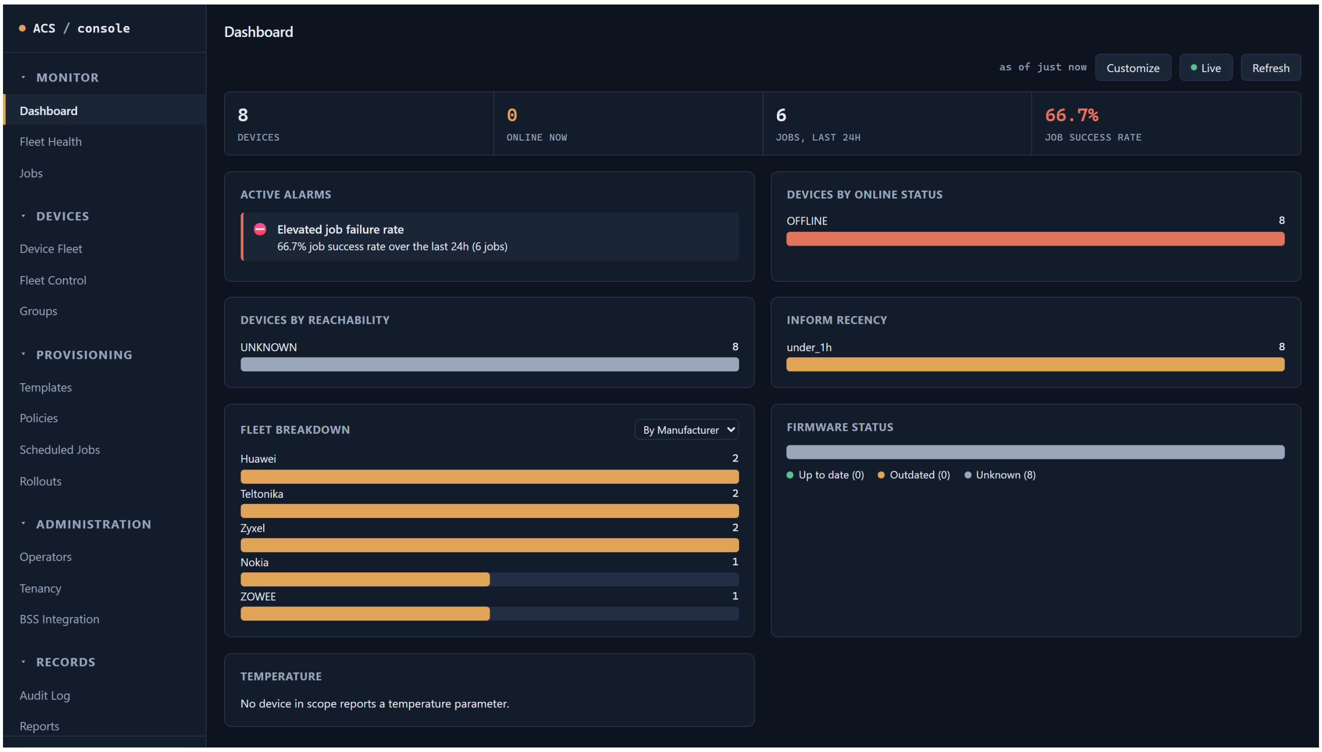
Figure A1 — The console sign-in



Sign-in is the only unauthenticated screen. The status line at the foot of the navigation shows whether the console can currently reach the API — the indicator that exposed blocker B6.

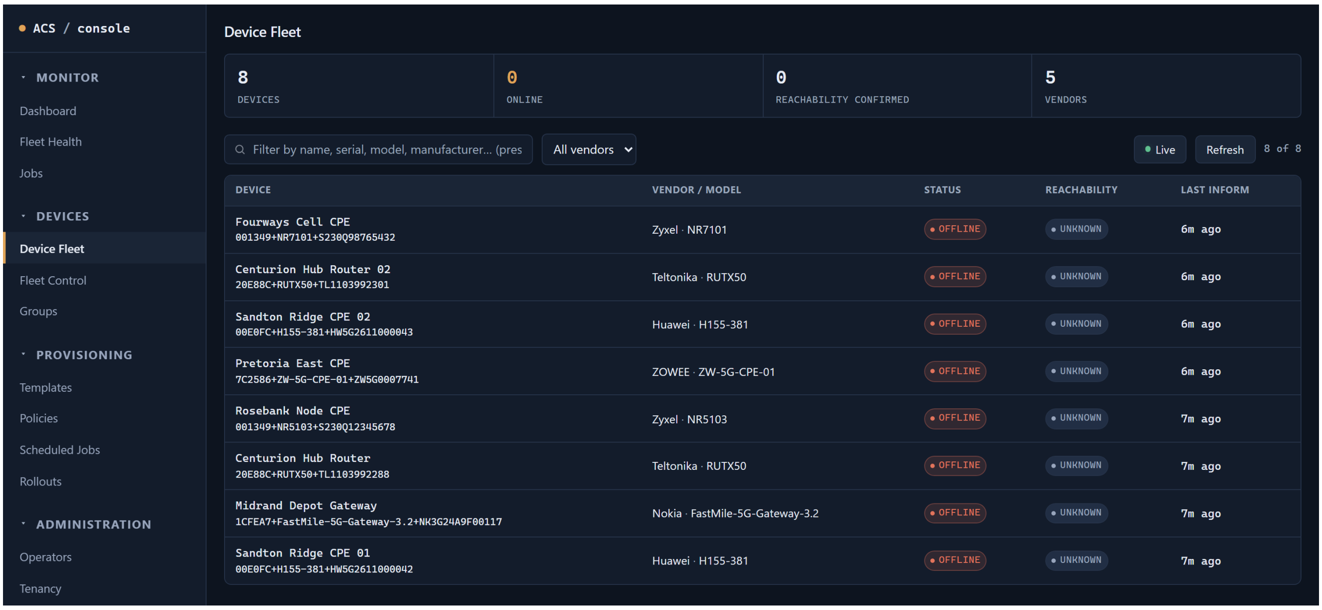
The daily view

Figure A2 — Dashboard



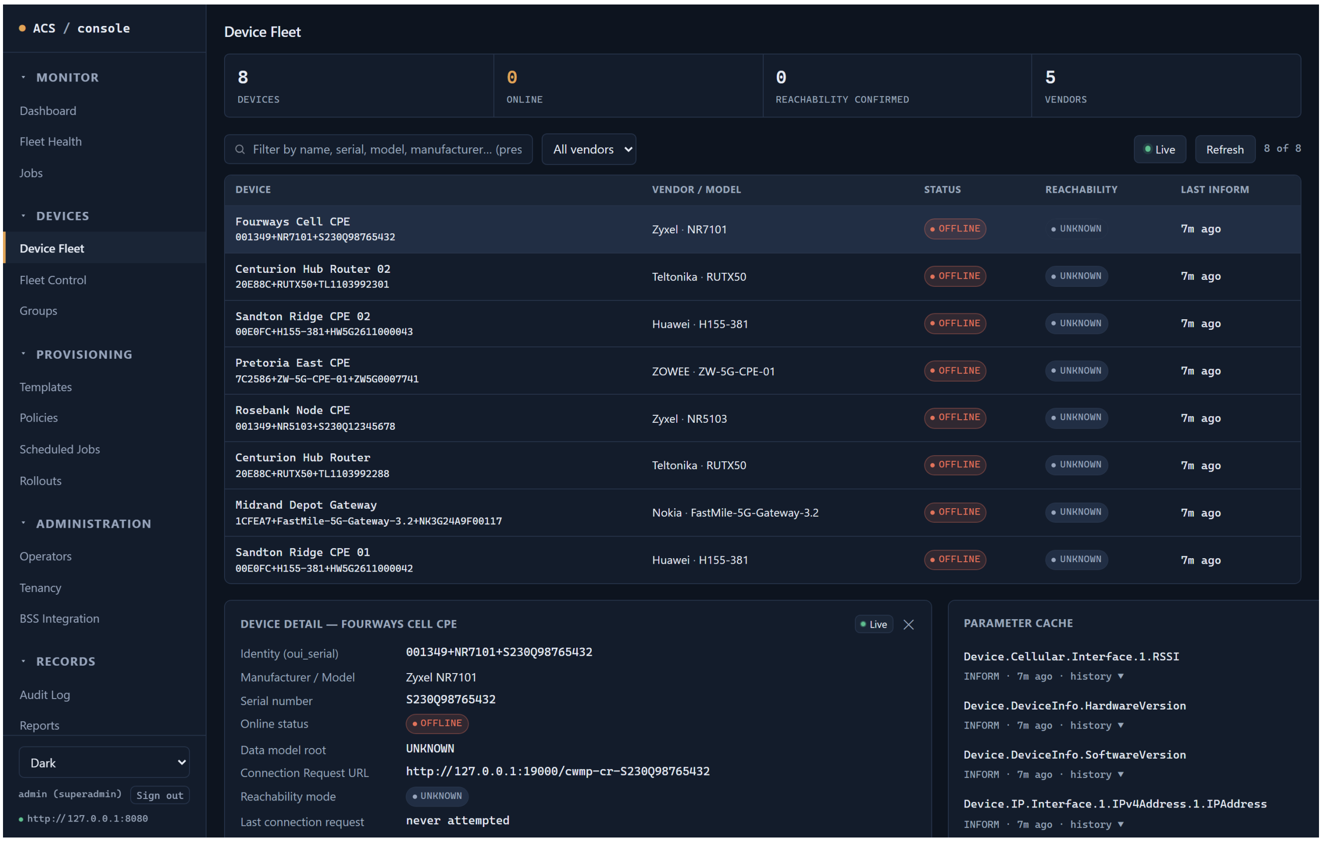
The operator's landing view: fleet state, recent activity and the panels an operator arranges for themselves. The layout is per-operator, so a NOC shift and a provisioning engineer are not forced to share one arrangement.

Figure A3 — Device Fleet



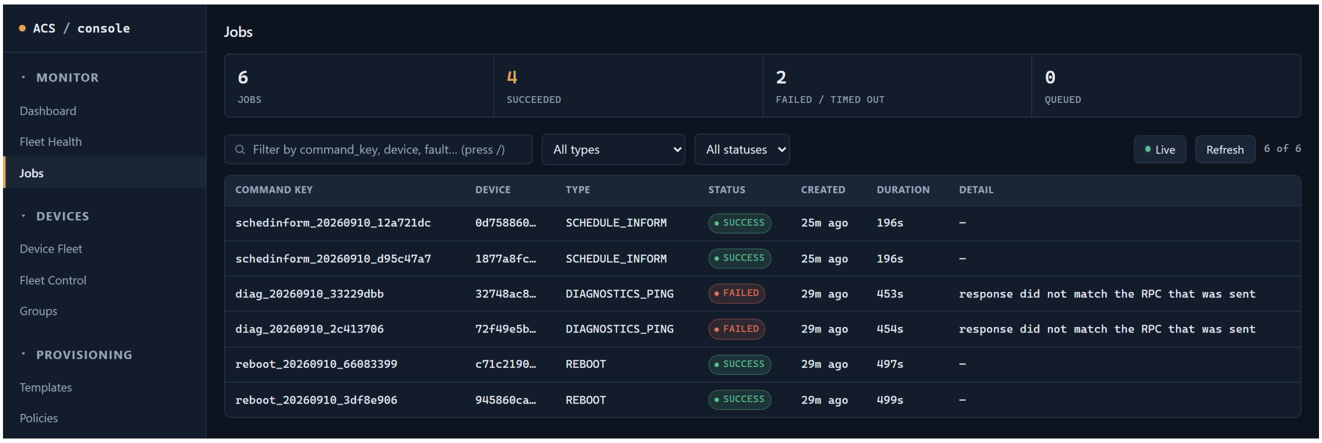
Eight devices across five vendors, each one having genuinely onboarded over TR-069 during this capture. Reachability reads UNKNOWN because the mock devices do not listen for a connection request — an honest reading rather than an optimistic default, and precisely the field that real-hardware qualification (B2) exists to turn green.

Figure A4 — One device in detail



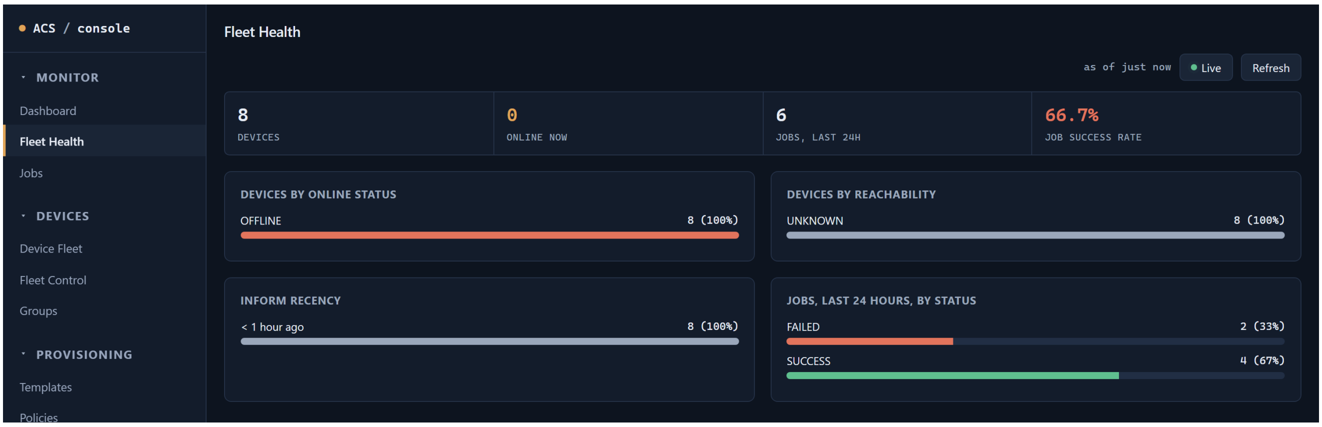
The engineer's working surface for a single subscriber: reported parameters, history, and the actions that fix it. The selection is carried in the address bar, so a device can be linked straight into a ticket.

Figure A5 — Jobs — what the platform tried, and how it went



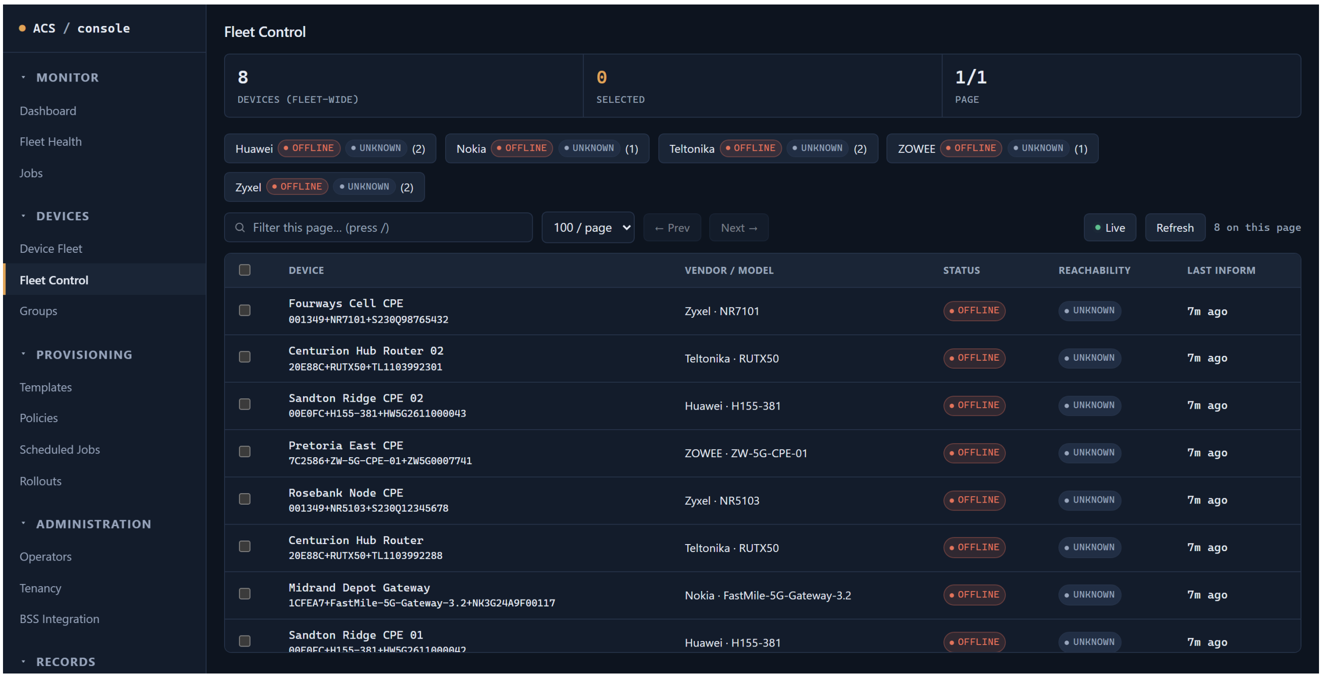
Six jobs from the live run: four succeeded, two failed. The failures carry their reason — “response did not match the RPC that was sent” — rather than a bare status, which is the difference between an engineer diagnosing a fault and guessing at it.

Figure A6 — Fleet Health



Fleet-wide condition rather than per-device detail: what is online, what has stopped checking in, and where failures are concentrating.

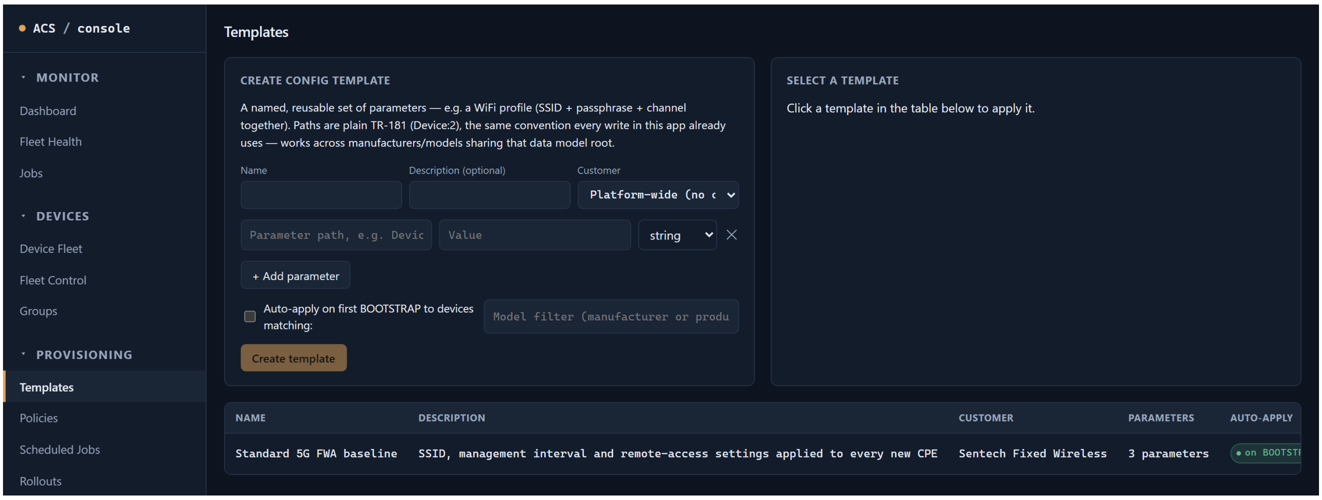
Figure A7 — Fleet Control



Bulk action across a filtered selection — the screen used to move many devices at once rather than repeating an action device by device.

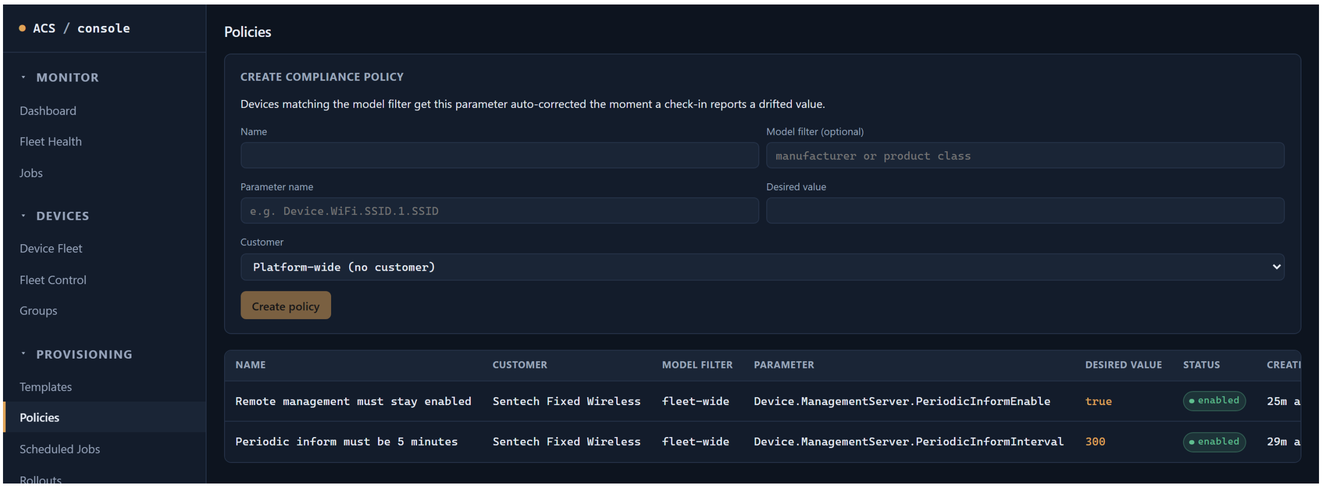
Provisioning: keeping the fleet to a standard

Figure A8 — Configuration templates



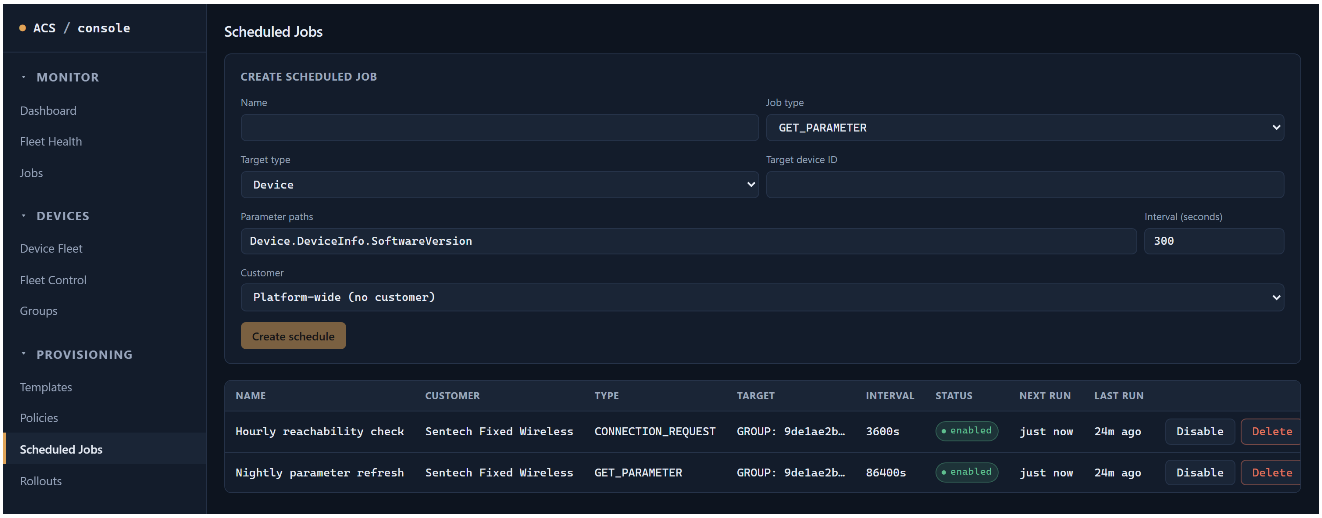
The template created live for this report. Marked to apply automatically to matching models, it pushed its settings to two devices on their next check-in with no operator involvement.

Figure A9 — Compliance policies



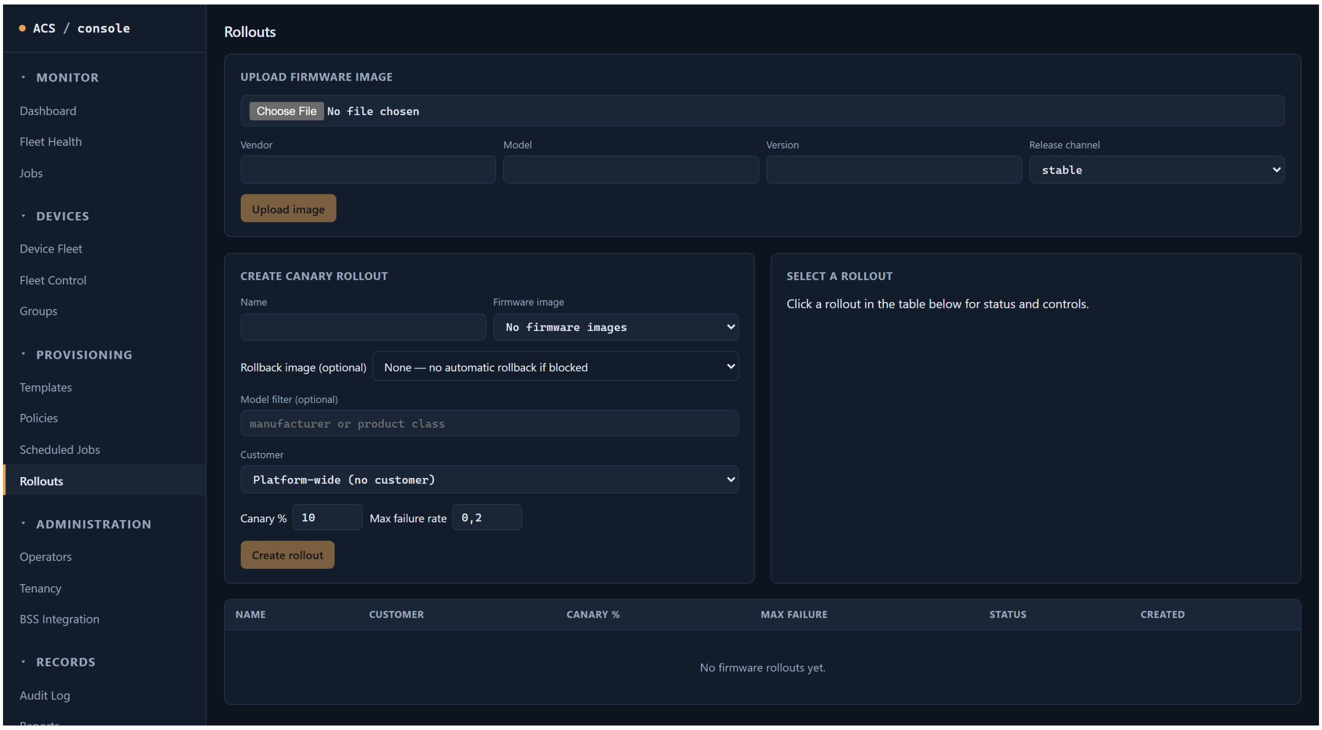
A rule checked on every device check-in. Drift shows as non-compliance continuously, rather than waiting for somebody to run a report.

Figure A10 — Scheduled jobs



Recurring work — a nightly parameter refresh and an hourly reachability check, both created live against the customer-scoped group. The platform enforces a minimum interval and re-checks tenancy at each firing, not only when the schedule is created.

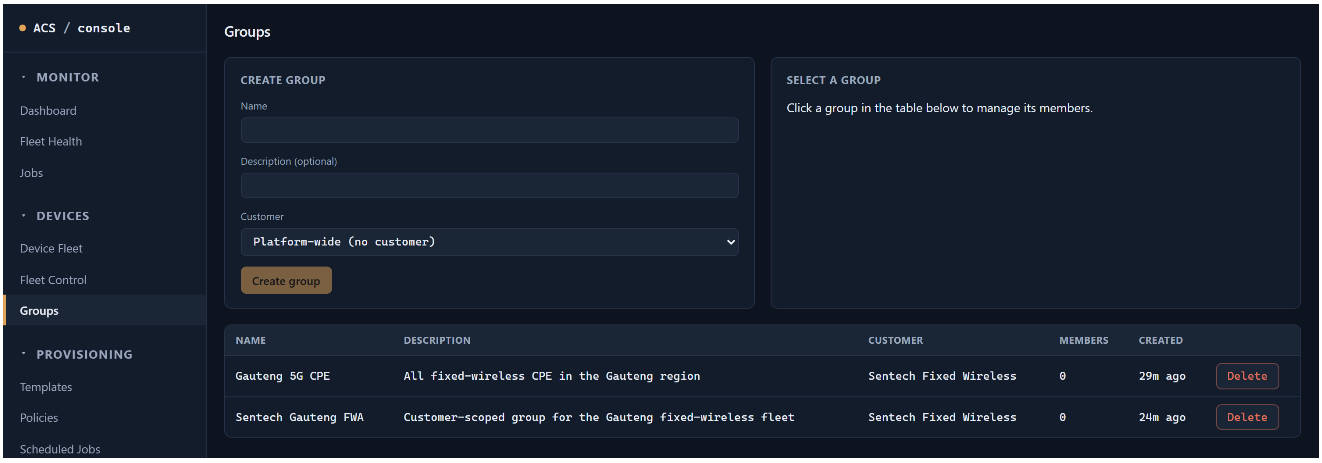
Figure A11 — Firmware rollouts



Staged rollout with a canary percentage and a maximum-failure-rate brake, so a bad image stops itself rather than continuing into the fleet.

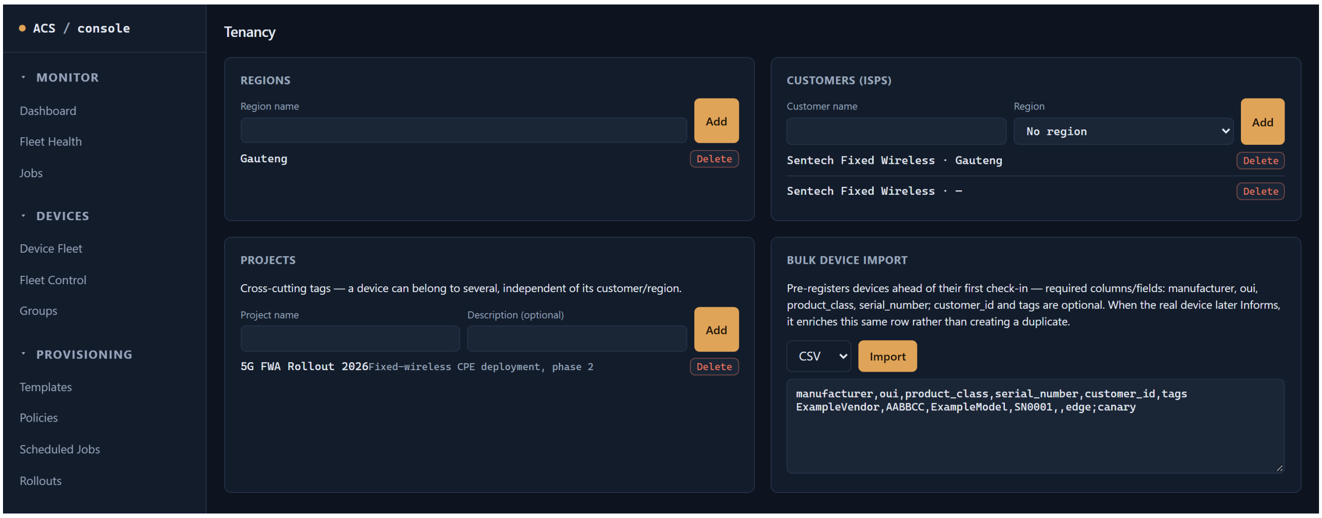
Organising the estate

Figure A12 — Groups



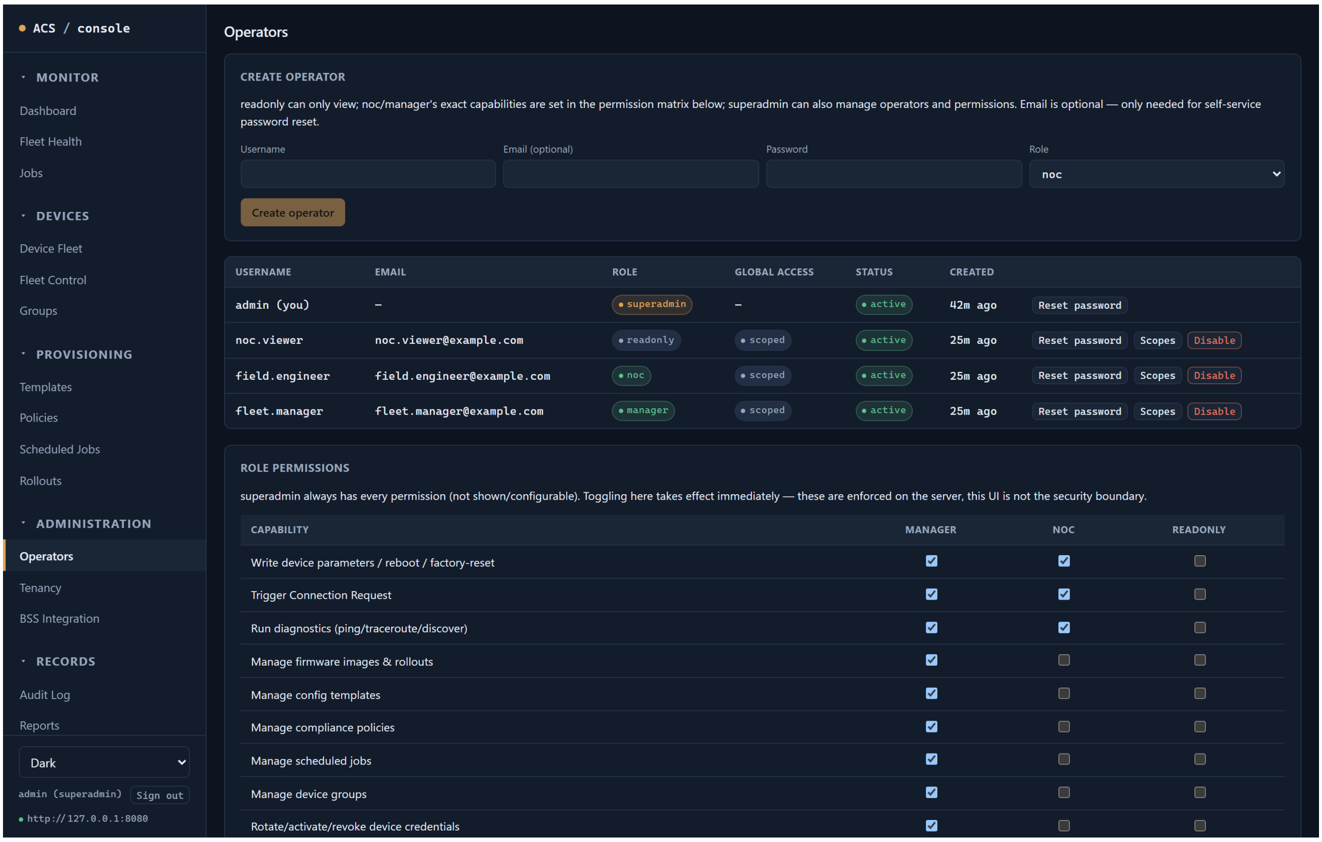
Devices gathered into a customer-scoped group, which is what bulk actions and scheduled jobs target. A scoped operator cannot create a group outside their own customer.

Figure A13 — Tenancy



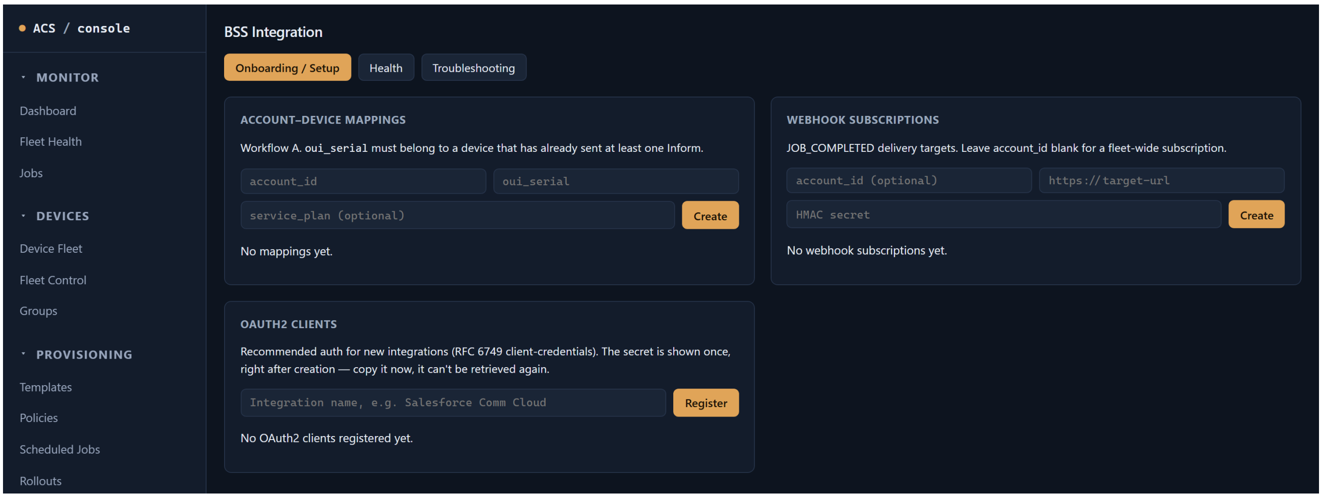
The region and customer hierarchy every other screen is scoped against. This is the structure that makes one deployment serve several customers without them seeing each other.

Figure A14 — Operators and permissions



Four permission tiers with a curated permission matrix. The three additional operators shown were created live through the API. An operator with no tenancy scope sees no devices at all — the default is deny.

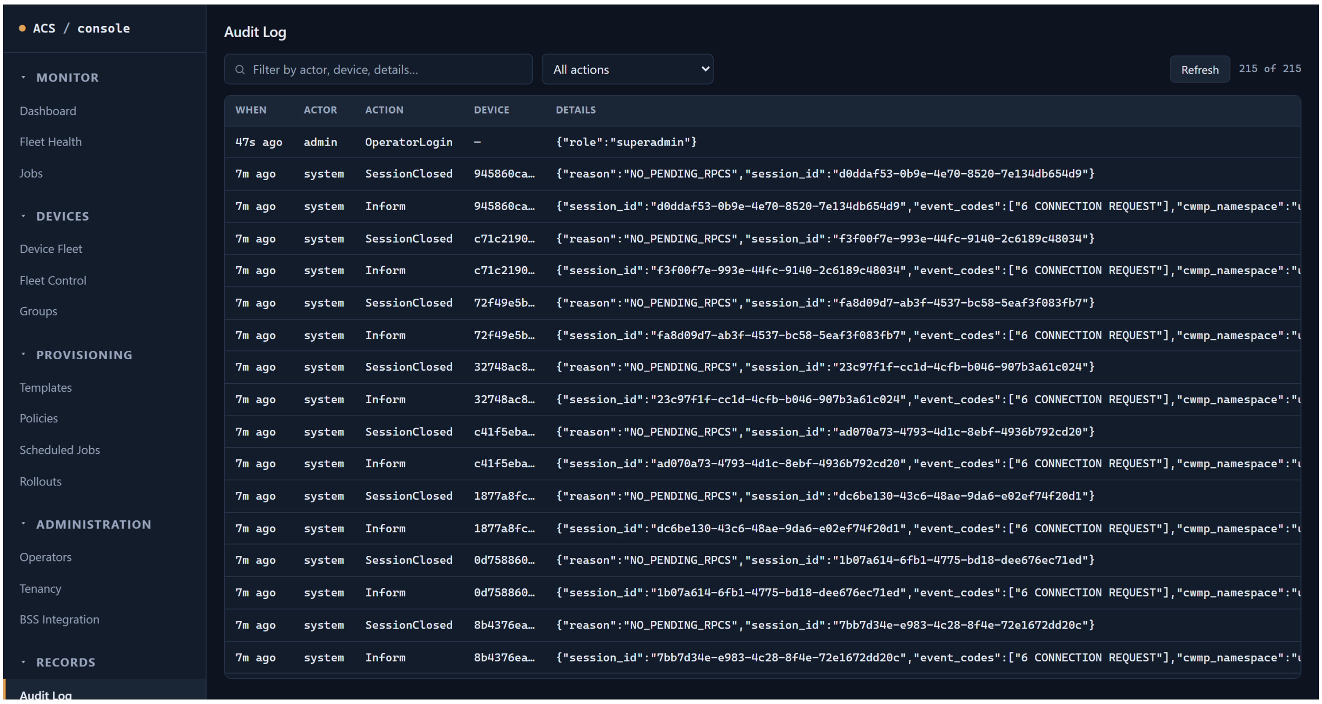
Figure A15 — BSS integration



The business-system contract, with its own troubleshooting panel — mapping lookup, authentication check, job status and order dispatch — so an integration fault is diagnosed inside the product.

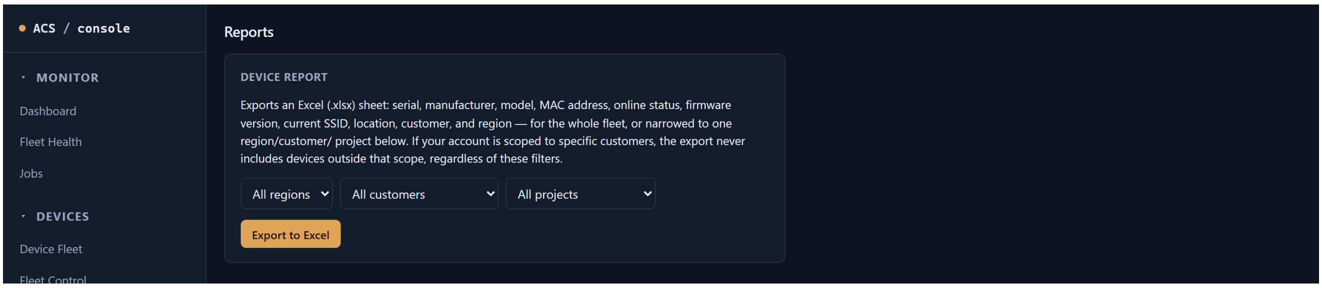
Records and oversight

Figure A16 — Audit log



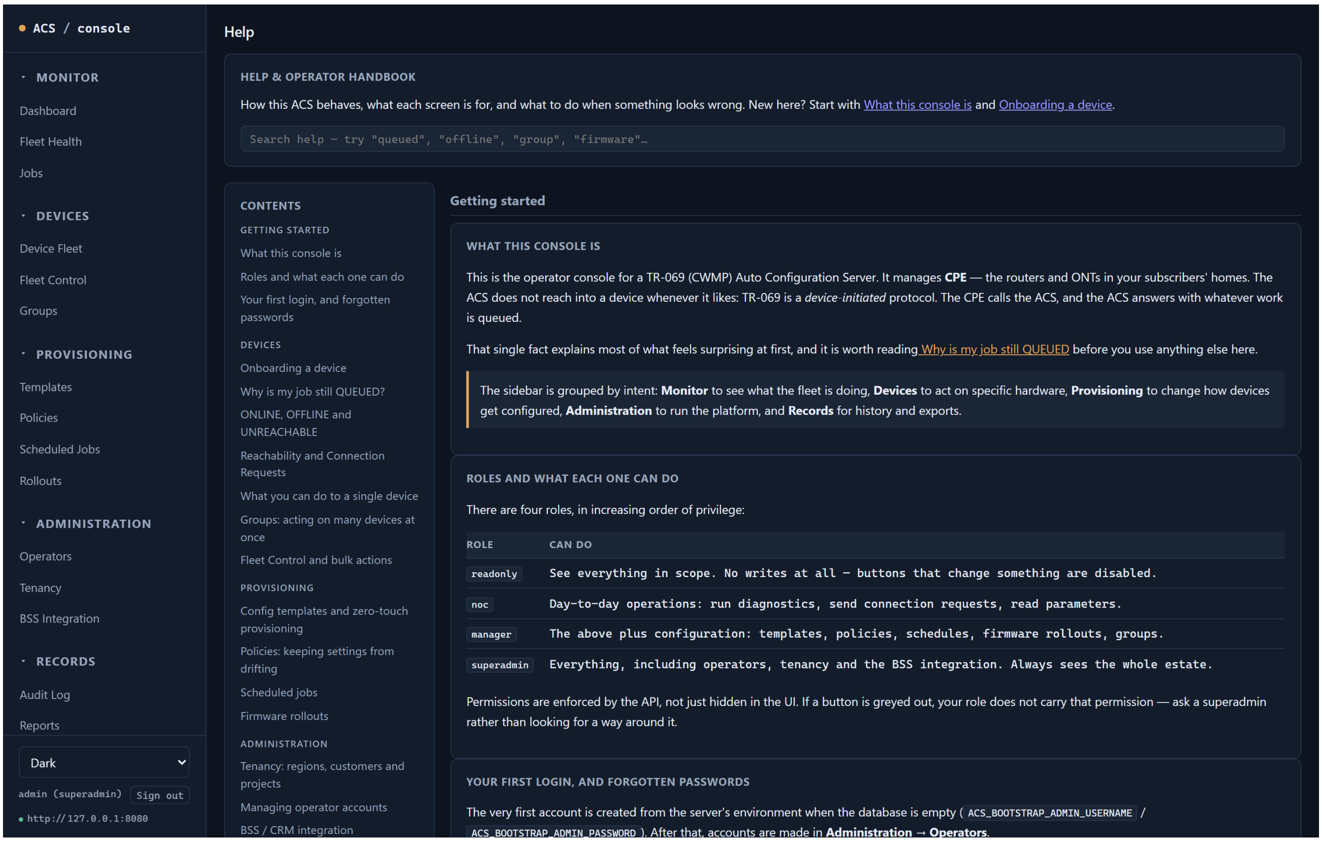
Every privileged action, with the operator, the target and the time. Terminal sessions opened to a device are recorded here too.

Figure A17 — Reports



Spreadsheet export for the people who work in one, rather than requiring them into the console.

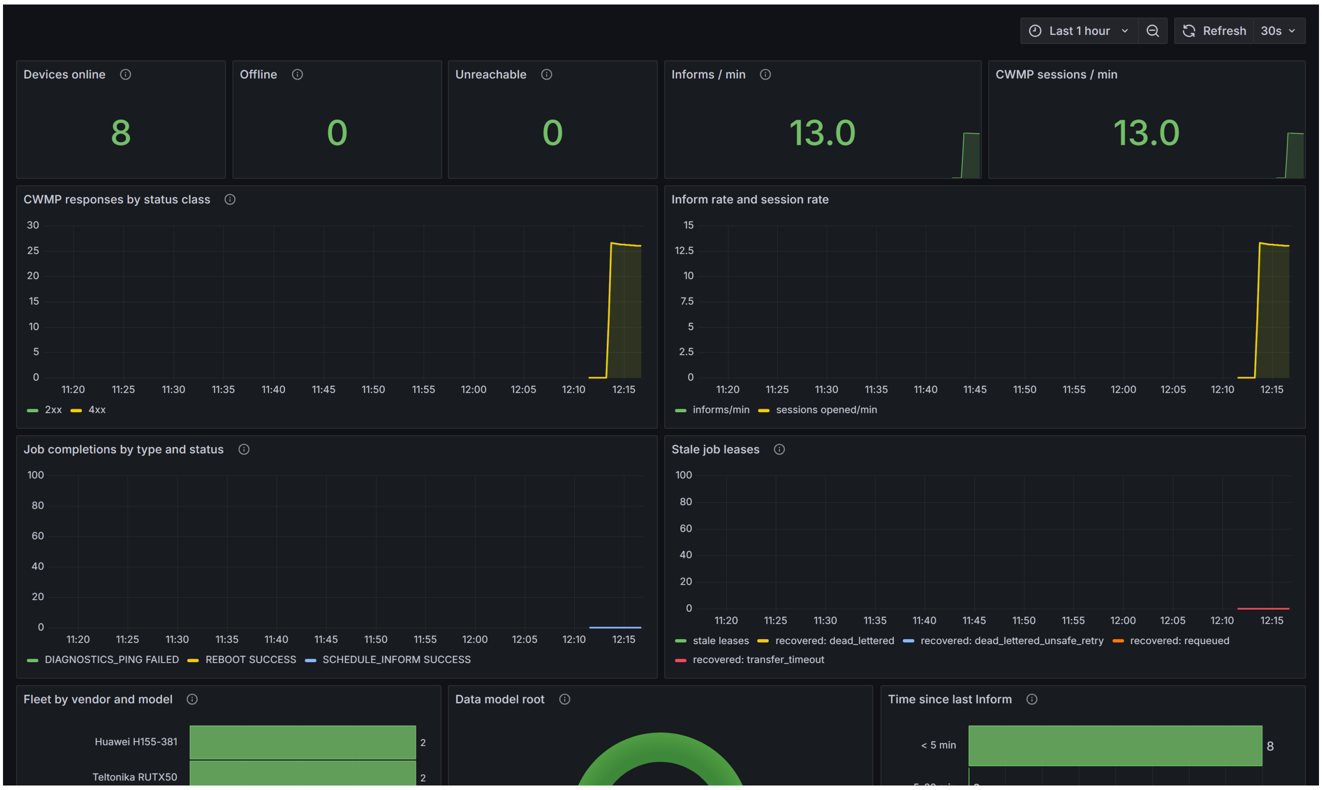
Figure A18 — The in-app handbook



Documentation carried inside the product. It matters for UAT: a tester who can answer their own question does not file a defect that turns out to be a question.

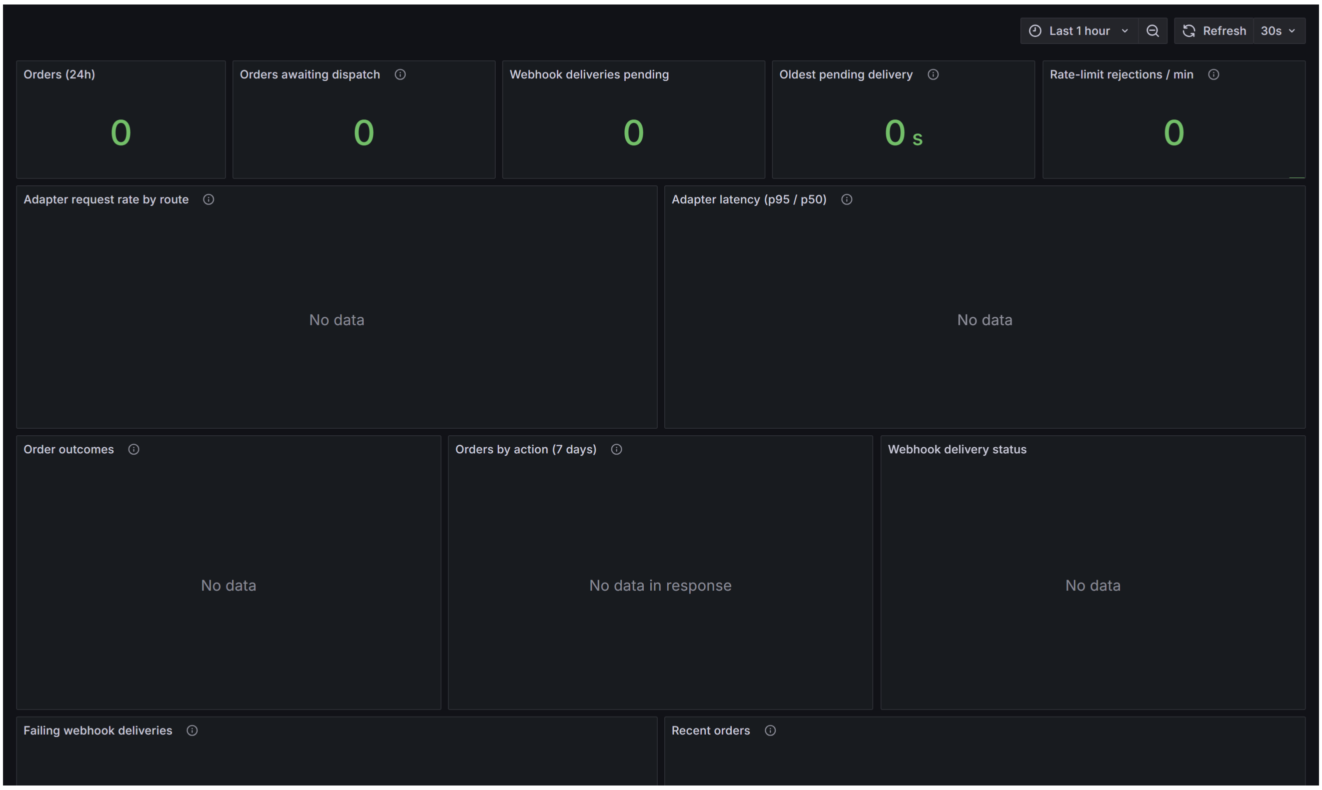
Monitoring: the platform's own health

Figure A19 — CPE Fleet dashboard



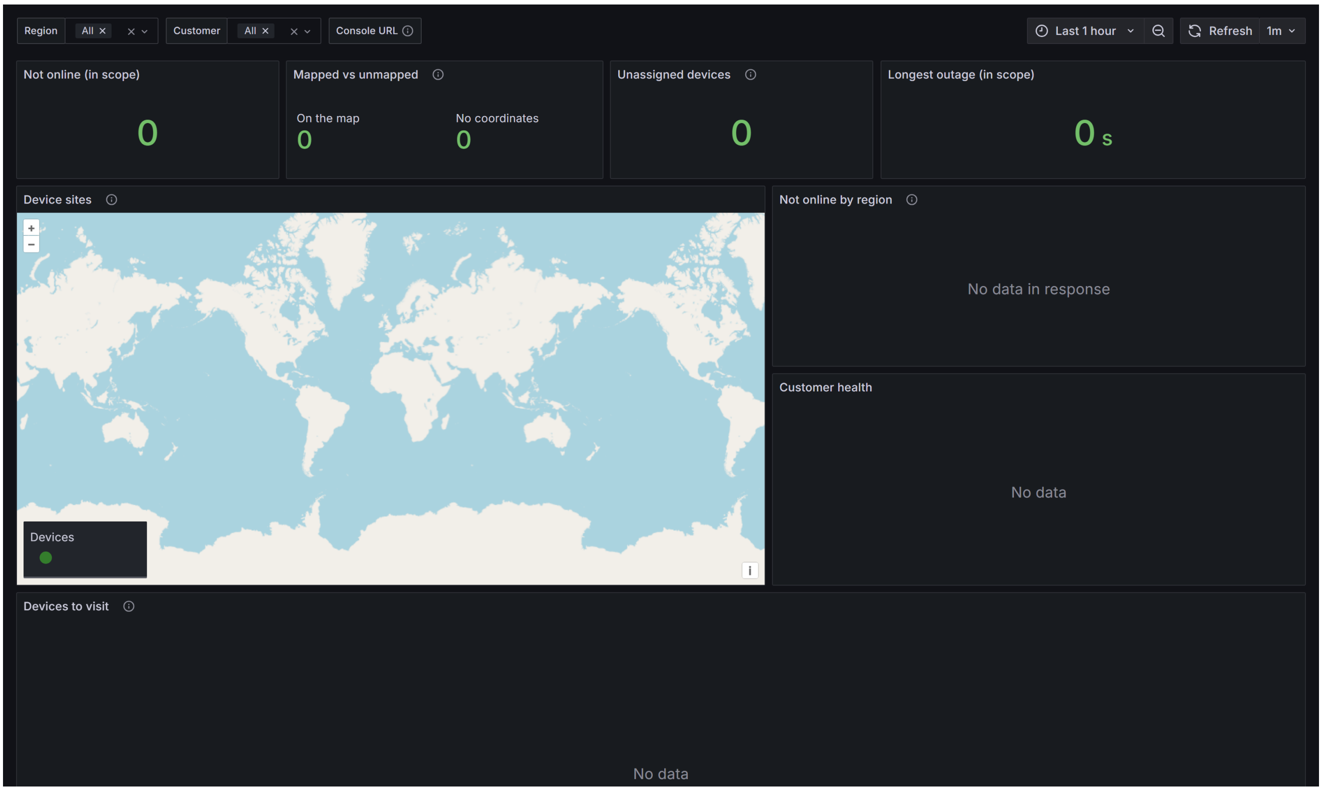
Live data from this report's own run: eight devices online, thirteen check-ins a minute, and job completions broken out by type and outcome — the two failed diagnostics and the successful reboots and schedule-informs are all visible here. The stale-lease panel is the queue's own safety net reporting on itself.

Figure A20 — BSS Integration dashboard



Order and webhook throughput for the business-system path. Quiet here because no orders were dispatched during the capture — the panels are wired and the adapter is being scraped.

Figure A21 — Tenancy and Site Locator dashboard



The relational view — customer, site, serial, last check-in — deliberately served from the database rather than the metrics store, through a read-only role whose grants exclude every column that could carry a secret.